

Progress Report 4

Student Name: Reubin Chatta

Student ID: 300394193

Project: Smart Door Anomaly Detection & Cybersecurity for Elderly Care

Course: CSIS 4495 – Applied Research Project, Section 3

Reporting Period: March 10 – March 16, 2026

Report Date: March 16, 2026

Work Date/Hours Logs

Date	Number of Hours	Description of work done
Mar 14, 2026	2.5	Added body-visibility check to LSTM detector in <code>fall_detection_trained.py</code> — counts non-zero keypoints; if < 50% visible, returns early with “Body not fully visible — move back from camera” to avoid false falls when only face/partial body in frame
Mar 15, 2026	3.5	Updated Falls dashboard UI: added Latest Fall Confidence bar (0–100% with Green/Amber/Red colors), visibility warning banner when “Body not fully visible”, updated <code>confidenceColor</code> rules; modified <code>fall_detection_camera.py</code> to POST visibility warnings (throttled) and fixed LSTM reset (R key); added JSDoc to <code>api.ts</code>
Mar 16, 2026	2.5	Created <code>test_fall_flow.sh</code> with three-terminal testing instructions; verified end-to-end flow (backend, LSTM detector, frontend); report writing Team Meeting
Total Hours	8.5	

Summary Description of work done during this reporting period

This week, I completed Phase 2 fall detection enhancements and dashboard improvements. Eric had already implemented the LSTM-based fall detector trained on the UR Fall Detection dataset; my tasks were to add a body-visibility safety check and update the Falls dashboard UI.

Body-visibility check: I added logic in `LSTMFallDetector.process_frame()` to count how many of the 66 pose keypoints are non-zero. If fewer than 50% are visible (e.g., when the camera sees only a face or partial body), the detector returns early with `is_fall=False` and `reason="Body not fully visible – move back from camera"` instead of running LSTM inference. This prevents false fall triggers when the person is too close to the camera.

Falls dashboard UI: I enhanced the Falls page with a “Latest Fall Confidence” bar showing 0–100% with color rules (Green < 0.5, Amber 0.5–0.8, Red > 0.8). I added a yellow visibility warning banner that appears when the latest event’s description contains “Body not fully visible” — prompting caregivers to ask the person to step back. The camera script was updated to POST visibility warnings to the API (throttled to once per 5 seconds) so the dashboard can display them. I also fixed the R key reset for LSTM mode so it clears the frame buffer and cooldown correctly.

Testing flow: I created `test_fall_flow.sh` with instructions for running the backend, LSTM fall detector, and frontend in three terminals, plus verification steps for falls, confidence bar, and visibility banner.

The integration ensures caregivers can monitor LSTM fall confidence, see visibility warnings when the camera cannot see the full body, and avoid repeated false falls in those situations.

Repo Check in of Implementation completed

The files I have checked in the repo are as follows:

- `Implementation/models/fall_detection_trained.py` — Body-visibility check; Keras model loading workaround for version compatibility
- `Implementation/frontend/lib/api.ts` — JSDoc for `FallEvent` (`anomaly_score`, `description`)
- `Implementation/frontend/app/falls/page.tsx` — Latest Fall Confidence bar, visibility warning banner, updated `confidenceColor`
- `Implementation/scripts/fall_detection_camera.py` — POST visibility warnings (throttled), LSTM reset fix (R key)
- `Implementation/scripts/test_fall_flow.sh` — Testing flow instructions

All changes committed via branch `reubinv2` with three commits.

AI Use Section

AI Tool	Version / Account Type	Specific feature for which the AI tool was used	Value Addition
Cursor	Composer / Agent	Code generation for body-visibility check, Falls UI components, camera script logic,	Customized for our

(Composer /Agent)		Keras model loading workaround	FallResult structure and API contract; ensured throttling and reset logic match LSTM detector attributes
ChatGPT	Pro Tier	Report text and structure suggestions	Adapted wording for project context and course requirements

Appendix: Prompt History

Prompt 1:

"In a fall detection script, I need to skip LSTM inference when fewer than half the pose keypoints are visible. How should I count non-zero values and return early?"

Prompt 2:

"I'm building a Falls dashboard in React. I need a confidence bar that changes color by range (e.g. green under 50%, amber 50–80%, red above 80%). What's a clean way to do that with Tailwind?"

Prompt 3:

"The camera script POSTs to the API on every frame when the body isn't visible, which floods the DB. How can I throttle it so it only POSTs at most once every 5 seconds?"